

LeetCode 363: Max Sum of Rectangle No Larger Than K

De-Show, Dunzo | Problem Q214

Approach 1: Bruteforce

The most straightforward approach involves identifying every possible rectangle. We can calculate the sums efficiently using a 2D Prefix Sum array. We apply row and column offsets to iterate over all possible sub-rectangles.

Method: Prefix-Sum + Row Offset + Column Offset

Time Complexity: $O(n^2 m^2)$

C++ Code (Approach 1)

```

class Solution {
public:
    int maxSumSubmatrix(vector<vector<int>>& matrix, int k) {
        int n=matrix.size();
        int m=matrix[0].size();

        int ans=INT_MIN;
        vector<vector<int>> pref(n,vector<int>(m));
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                pref[i][j]=matrix[i][j];

                if(i>0){
                    pref[i][j]+=pref[i-1][j];
                }

                if(j>0){
                    pref[i][j]+=pref[i][j-1];
                }

                if(i>0 && j>0){
                    pref[i][j]-=pref[i-1][j-1];
                }
            }
        }

        for(int h=1;h<=n;h++){
            for(int w=1;w<=m;w++){
                for(int roff=0;roff+h<=n;roff++){
                    for(int coff=0;coff+w<=m;coff++){
                        int r1=roff;
                        int c1=coff;
                        int r2=roff+h-1;
                        int c2=coff+w-1;

                        int sum=pref[r2][c2];

                        if(r1>0){
                            sum-=pref[r1-1][c2];
                        }

                        if(c1>0){
                            sum-=pref[r2][c1-1];
                        }

                        if(r1>0 && c1>0){
                            sum+=pref[r1-1][c1-1];
                        }

                        if(sum<=k){

```

```

        ans=max(ans, sum);
    }
}
}
return ans;
};

```

Approach 2: Beautiful Optimization

Instead of fixing the *height* and *width* of the rectangle directly, we can optimize by fixing the **top row** and **bottom row**.

By fixing the boundaries vertically, every column gets compressed into a **single number** (the sum of the elements in that column between the top and bottom rows). This effectively reduces our 2D problem into a 1D problem!

Matrix Reduction Example

Consider the following matrix:

1	2	-1
3	-2	4
5	1	-3

Let **top = 0** and **bott = 1**. We sum the elements vertically for these rows. The columns become:

Col 0 (1+3)	Col 1 (2-2)	Col 2 (-1+4)
4	0	3

Now, the 2D problem becomes: **Find the max subarray sum $\leq k$ in the 1D array [4, 0, 3].**

Note: A standard sliding window will NOT work here due to the presence of negative numbers.

1D Array Optimization (Using Set)

Time Complexity: $O(n^2 m \log m)$

Suppose after fixing the top and bottom rows, we get the 1D array: [4, -2, 3, 1].

A bruteforce approach on this 1D array would still take $O(m^2)$. We can optimize this by thinking in terms of **prefix sums**.

The prefix sums for [4, -2, 3, 1] are [4, 2, 5, 6]. Adding a 0 at the beginning to represent an empty prefix, we get:

0	4	2	5	6
---	---	---	---	---

Mathematical Formulation

Let the current prefix sum be P . We want to find a previous prefix sum x such that the subarray sum is $\leq k$.

$$P - x \leq k \quad \rightarrow \quad x \geq P - k$$

Among all the previously seen prefixes that satisfy this condition, we need to choose the **smallest** x so that the resulting subarray sum ($P - x$) becomes as large as possible (maximum).

Lower Bound Example

Suppose our stored previous prefixes are [0, 3, 7, 11, 18].

If we currently need $x \geq 8$, the smallest available valid prefix is **11**. We can efficiently find this in $O(\log m)$ time by using the LOWER_BOUND function on a Balanced Binary Search Tree (std::set in C++).

C++ Code (Approach 2)

```
class Solution {
public:
    int maxSumSubmatrix(vector<vector<int>>& matrix, int k) {

        int n = matrix.size();
        int m = matrix[0].size();
        int ans = INT_MIN;

        for (int top = 0; top < n; top++) {
            vector<int> col(m, 0);

            for (int bott = top; bott < n; bott++) {
                for (int j = 0; j < m; j++) {
                    col[j] += matrix[bott][j];
                }

                set<int> st;
                st.insert(0);
                int pref = 0;
                for (int j = 0; j < m; j++) {
                    pref += col[j];
                    auto it = st.lower_bound(pref - k); class="cmt"> // we need pref-x>=k ,
                                                         class="cmt"> // needs to be as small as possible

                    if (it != st.end()) {
                        ans = max(ans, pref - *(it));
                    }

                    st.insert(pref);
                }
            }
        }

        return ans;
    }
};
```